A Document-Stack Methodology for Agentic Software Development

Bjørn Remseth
Email: la3lma@gmail.com

***Abstract*—This paper describes a personal methodology the author has used for driving software and product development with AI agents. The method organises work as a stack of documents – a concept document, a product requirements document, an optional architecture document, and an execution plan expressed as a graph of use cases in a style inspired by Alistair Cockburn. Each use case captures pre- and post-conditions in the spirit of Hoare triples, an explicit set of actors and steps, and the observable evidence required to believe the task has actually been completed. The execution plan is the topological ordering of these use cases. A human can engage at any level of granularity, from fully delegating execution to working hand-in-hand with the agent on each step. The approach is deliberately implemented with a single capable agent rather than a multi-agent orchestration.**

***Index Terms*—agentic software development, use cases, Cockburn use cases, product requirements, Hoare triples, execution planning, AI-assisted development**

`version: 2026-06-29-10:02:20-CEST-6fc5730-main`

Note: AI was used as a co-writing tool; responsibility for the final text and references is mine.

I. Introduction

This paper describes a method the author has been using to drive software and product development together with AI agents. The intention is twofold: to write the method up in a paper-style document, and to accompany it with a public GitHub repository containing the prompts, plots and supporting artefacts that the method relies on in practice.

The method centres on a stack of documents. It starts with an idea that is more or less vaguely formulated – often just a sentence or two written in the author's own words, plus whatever notes or concrete references seem likely to improve it. From that seed, the AI is asked to produce the documents that, taken together, describe the system and the path to building it. Figure 1 sketches the resulting loop as a use-case diagram.

II. The Document Stack

A. Concept Document

The first document in the stack is the concept document. It describes the overall idea and places it in context. It often includes some more or less precise success criteria. The aim is to put the idea into a form that makes sense for at least the author, but ideally also for someone else who picks it up later. The concept document is usually a little verbose, because it is not meant to be a tidbit; it is meant as an in-depth description of what is going on. It is not, however, a product description.

B. Product Requirements Document

The next paper in the stack is a Product Requirements Document (PRD). The PRD is more descriptive of what is going to come out of the work. It describes the known goals, and outlines the overall architecture, typically with the components that can be used. Where there is uncertainty, that uncertainty is carried into the PRD so that subsequent work is explicitly aimed at reducing it. The PRD is a mix of implementation ideas and necessities, but its dominant viewpoint is the product as seen by a consumer.

C. Architecture Document

It may or may not be necessary to create a separate architecture document. Sometimes it is sufficient to include the architectural ideas inside the PRD. Other times it is worth separating them. The architecture document is usually not very complicated – and if it is, that is a signal that the idea is not yet simple enough.

The architecture document starts with a title and a diagram showing which components are part of the system and what they are basically doing when they interact. It continues with a brief description of what the system does and how the components achieve their goals by interacting, and then typically a description of a happy-day scenario. Edge cases are usually not the focus at this point; the goal is to nail the happy-day scenario first. The same happy-day bias carries through into the use cases described in the next section.

III. The Execution Plan and Simple Agentic Use Cases

After the conceptual documents, the next artefact is the execution plan. This is a separate document that lays out the steps necessary to get from the initial condition – in which we have only the context described by the documents above – to the end condition, in which the system is operational.

The plan decomposes the work into sub-tasks. The author models these sub-tasks as *simple agentic use cases* – "agentic" because they are written to be executed by an AI agent, and "simple" as a deliberate constraint on how much each one is allowed to contain. The style is inspired by Alistair Cockburn's work on use cases [1]: a balanced mix of formal and informal that the author finds particularly useful in this setting.

Each simple agentic use case has the following fields:

- **Name**
- **Goal**
- **A very brief description**

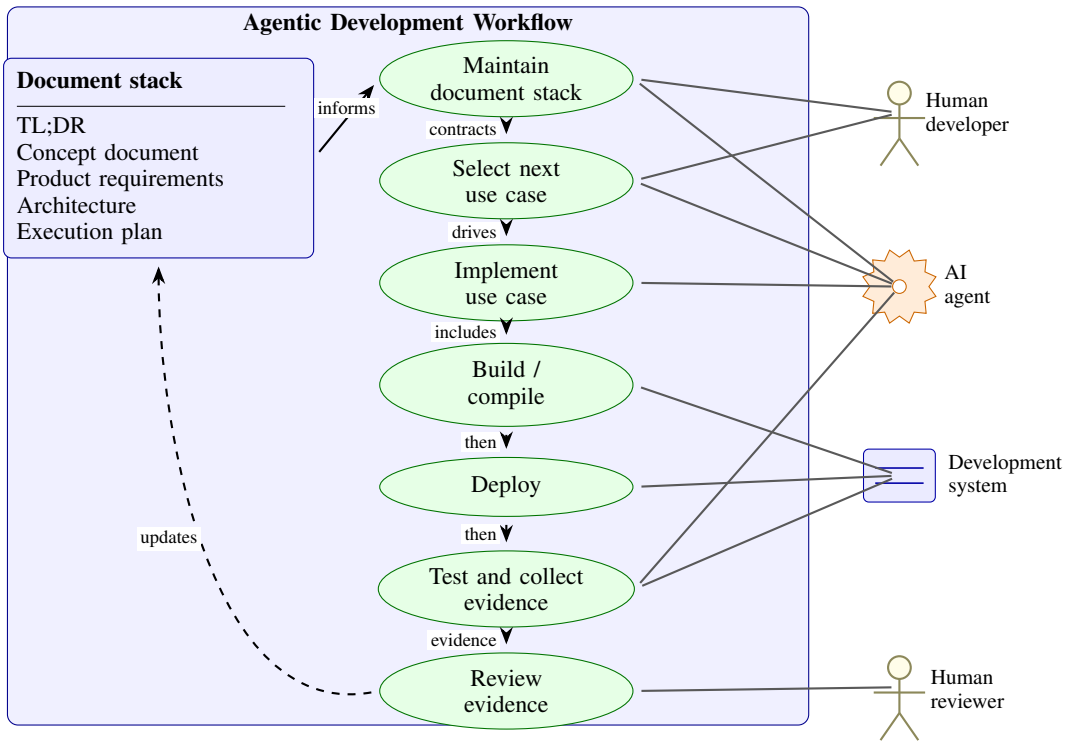

Fig. 1. *The method as a UML-style use-case diagram: a human developer and an AI agent maintain the document stack, choose and execute use cases, operate the development system, and feed evidence back into review and stack updates.*

- **Actors**: the entities interacting to get the work done. There may be only one actor, but certainly not zero.
- **Pre-conditions**: functional conditions that must be in place before this use case can be implemented or executed.
- **Post-conditions**: the conditions that are true after the use case has been run.
- **Steps**: the actions that take the pre-conditions into the post-conditions.
- **Observables**: the evidence a developer would want to inspect before believing that the use case has been faithfully completed.

One reason for liking this structure is that it is not just reminiscent of, but actually uses, Hoare triples [2] to describe the functional behaviour of a use case. This is theoretically a good idea and can be extended toward formal proof if and when that is desired. It is certainly in the right frame of mind for doing so – not necessarily always the right thing to do, but good to have as an option.

A. Observables and Evidence

The observables are the things that the developer wants to look at in order to believe that a sub-task has been faithfully completed. They can be logs, or screenshots produced by some kind of automated capture mechanism. The author's personal preference for web-based work is Playwright [3], but this varies with the domain. Robotics work, for example, will demand other forms of evidence. The non-negotiable part is that there must be evidence that the task is actually done, and that one can inspect that evidence after the fact.

B. Happy-Day Steps and Blocking on Errors

The steps of a use case describe the *happy day* only – the sequence of actions that takes the pre-conditions to the post-conditions when nothing goes wrong. Use cases are deliberately not littered with exception branches, error handlers and alternative flows. The reason is partly aesthetic and partly pragmatic: a happy-day description is easier to keep coherent with the rest of the stack, and it is what makes the Hoare-triple framing tractable.

When errors are actually encountered during execution, the response is to apply ordinary debugging techniques – read the logs, try the obvious fixes, narrow the problem down, retry. If debugging succeeds, the task continues toward its post-conditions and nothing about the use-case description needs to change. If debugging does not succeed, the task is marked as *blocked* (the red state described in Section IV) and a human is asked for help. Blocking is the explicit, first-class failure mode of the method, and it is preferable to silently inventing exception branches that no one has thought about.

There is a deeper reason for the happy-day-only convention, beyond aesthetics and tractability. The use cases in this method are *implementation* tasks, and implementation is something one undertakes only after the genuinely hard parts have already been figured out. If a task still contains substantial unknowns – algorithmic, scientific, design-level, or otherwise – then it is not really an implementation task at all. It belongs in a separate

research track, whose output is precisely the understanding that makes a subsequent implementation use case writable in happy-day form.

Once that division of labour is enforced, the happy-day description becomes a fairly good approximation of what will actually happen when the task is run. The hard parts have already been de-risked; what remains is the routine work of turning a known design into running code, with ordinary debugging absorbing the small surprises that inevitably show up along the way. In other words, a use case that *cannot* reasonably be written as a happy day is a signal that the work upstream of it is not yet finished – and the right response is to do that upstream work, not to bloat the use case with speculative branches.

IV. Execution as a Graph

Based on the functional decomposition, there is a natural dependency relationship between use cases: some are upstream and others downstream. To start a task, certain other tasks must already have run, otherwise the pre-conditions are not in place. The result is a directed graph.

The execution rule is straightforward: at any moment, consider the tasks whose predecessors have all been run, and among those choose the best task to do next.

A. How a Human Interacts with the Process

A human can inject themselves at any point. Sometimes, when the work is straightforward, the right move is to let the machine do all of it and be surprised at the result – then inspect the evidence and, if it is good, move on. In other cases, where the risk is high or the understanding is shaky, it is better for the human to work hand-in-hand with the agent, doing each step deliberately and inspecting it slowly.

In either case, the output is the same in kind: a clear description of what was done and why, how the work fits into the larger context, and an evidence trail showing that everything that was supposed to happen actually happened.

B. Tracking the Graph

Typically, the author asks the system to maintain a single document that lists all of the tasks and renders their dependencies as a graph. The author's preferred diagramming tool for this is Mermaid [4], but any graphing tool can be used. Clicking a node in the graph leads to the task description.

Sometimes the system is instructed to keep the task descriptions in GitHub as GitHub issues. Task hierarchies – epics and sub-tasks – are then codified with GitHub labels. The clickable nodes of the graph point to the corresponding GitHub issue. Once an issue has been completed and is part of a pull request, the node points to the PR instead, which shows both the run and the collected evidence such as screenshots.

A small worked example accompanying this paper is available as the Agentic Breakout example site at https://la3lma.github.io/agentic-breakout-example/, with the corresponding public repository at https://github.com/la3lma/agentic-breakout-example. It contains the document stack, the issue-to-PR graph, the lab notebook, evidence pages, and a playable static JavaScript game produced through this process. The example is intentionally modest: reimplementing classic Breakout is hardly a challenge for a contemporary programmer or a capable coding agent, so applying the full document stack to it is massive overkill for the game itself. The virtue of the example is that it is clean, familiar and easy to inspect; the same technique is not limited to classic arcade game reimplementations. A captured evidence frame from the running game is shown in Figure 2. That example was created using ChatGPT 5.5 Extra High, but the technique has also been tried with several other agents, including Opus 4.7. Better models tend to produce better results, but the method is not bound to a particular stratum of model. It works reasonably well with any model capable of program synthesis.

Other task-tracking systems work equally well – Jira, for example. At smaller scales, a single Markdown file is fine; at larger scales, a directory or even a hierarchy of directories, with one Markdown file (or one directory of evidence) per task, works well too.

C. Colour Coding of Graph Nodes

To make the state of the work legible at a glance, nodes in the dependency graph are colour coded according to their current lifecycle state:

- **Green – done**: the task has been completed and its evidence has been reviewed.
- **Yellow – ready to run**: all pre-conditions are satisfied; the task can be started.
- **Orange – running**: the task is currently in progress.
- **Blue – ready for review**: execution has finished and the evidence is awaiting human inspection.
- **Red – blocked**: the task cannot proceed, typically because a dependency is unmet or an external blocker is in the way.
- **Purple – deferred for later**: the task is intentionally postponed and is not under active consideration.

This colouring complements the structural dependency view: the graph shows what *can* be done, while the colours show what *is* being done and what is waiting on whom. In practice this makes it easy to scan the graph and find the next thing to pick up, or the part of the system that needs attention.

V. The Stack as a Living, Coherent Artefact

It is completely normal – and expected – for the document stack to mutate during the work. Features get added. Perspectives shift. Assumptions that looked sound in the concept document turn out to be wrong once an architectural detail is examined, or once a use case forces the PRD to be more specific. The stack is not a contract written at the start and frozen; it is a living artefact that evolves alongside the implementation.

The non-negotiable property is *coherence at all times*. When something changes, the change is not localised to a single document: it is propagated through the entire stack in the same edit. If the PRD gains a new feature, the concept document is

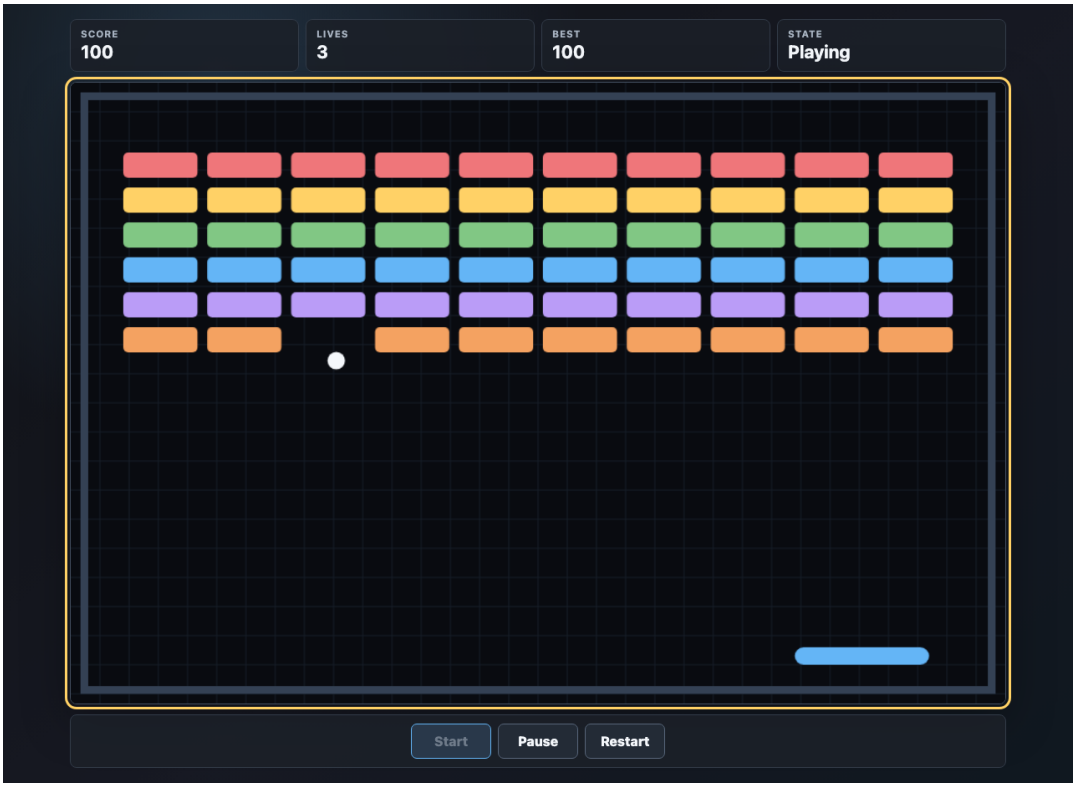

Fig. 2. *A Playwright-captured evidence screenshot from the Agentic Breakout example. The game is deliberately simple; the screenshot is included to show the kind of concrete, inspectable artefact the method asks the agent to produce while executing the document stack.*

updated to motivate it, the architecture document is updated to accommodate it, and the execution plan gains, removes or restructures the use cases needed to deliver it. If a use case discovers that a pre-condition is infeasible, that discovery flows back up the stack until the inconsistency is resolved.

The consequence is a strong invariant: at any moment in time, the stack is a coherent description of the work. At the start, it is a coherent description of what we *want*. At the end, it is a coherent description of what we *have*. There is no moment in between at which the stack is allowed to lie about either.

This invariant is one of the main reasons the method is workable with a single capable agent: the agent's job is not just to execute the next use case, but to keep the stack consistent as reality intrudes on the original plan.

VI. Working Documentation, and the TL;DR

It is worth being clear about what kind of documentation the stack actually is. It is *working documentation* – documentation for the process of building the artefact – and not documentation for the eventual users of the artefact. The concept, PRD, architecture, use cases and execution plan exist to drive and audit the work, not to onboard end users.

That said, the stack contains a lot of the raw material from which user-facing documentation can later be generated. Goals, success criteria, happy-day scenarios, component diagrams and use-case flows all map naturally onto sections of a user guide or an API overview. Producing user documentation from the stack is a worthwhile downstream step, but it is a distinct activity with a distinct audience.

One reasonable objection to the method is that this is a lot of text and a lot of documents for something that is, taken as a finished system, often not all that complex. The objection is fair; the author agrees. The remedy is to add one more document to the stack: a TL;DR.

The TL;DR is a one- or two-pager whose sole job is to summarise, in brief, what all the other documents are saying. It states the idea, the intended outcome, the shape of the system, and the broad plan to get there – nothing more. A reader who only has a few minutes should be able to read the TL;DR and walk away with an accurate gist. A reader who wants to dig in can then follow the TL;DR's pointers down into the full concept document, the PRD, the architecture and the use cases.

The TL;DR is subject to the same coherence invariant as the rest of the stack: when the stack changes, the TL;DR changes with it. Its job is not to be a frozen elevator pitch from the start of the project, but a faithful, always-current summary of where the work actually stands.

VII. Related Work

The method described here sits at the intersection of two literatures.

The first is the long-running tradition of use-case-driven software development. The shape and field structure of the simple agentic use cases described here borrow directly from Alistair Cockburn's work on effective use cases [1], which has been the de facto practitioner reference for over two decades. The use of explicit pre- and post-conditions to give each use case a precise functional contract is, in turn, an application of Hoare's axiomatic semantics for computer programming [2]: a simple agentic use case is, in essence, a Hoare triple wrapped in enough scaffolding to be useful to humans and AI agents alike.

The second is the rapidly growing literature on language-model-based agents. The pattern of interleaving reasoning steps with concrete actions in an external environment was crystallised by ReAct [5] and has since become foundational for modern agentic systems. At the more practitioner-facing end, Anthropic's overview of agent design patterns [6] argues for composable, auditable agents over elaborate multi-agent orchestrations – a position the present method endorses in the strongest possible way by using a single capable agent rather than a swarm.

The contribution of this paper is not in either of those areas individually, but in the deliberate combination: borrowing the contract-and-evidence rigour of the use-case tradition, applying it at the granularity required to drive an LLM-based agent, and binding the whole arrangement together with a coherence invariant on the document stack.

VIII. Discussion

A. One Agent, Not Many

A natural question is whether this approach involves orchestrating a swarm of specialised agents. It does not. The method is implemented with a single, capable agent that produces and operates on the entire document stack. The structure – documents, use cases, dependency graph, observables – comes from the artefacts and the process, not from the topology of the agent system.

B. Why This Works

The document stack does several things at once. It forces the idea to be progressively sharpened: a vague seed becomes a concept, becomes a PRD, becomes an architecture, becomes a graph of small, checkable use cases. The Hoare-triple framing of pre- and post-conditions gives each unit of work a precise contract, which makes the work amenable to both delegation and inspection. The observables enforce honesty: a task is not done until its evidence exists and has been seen.

IX. Conclusion

The method described here is, in essence, a way of letting a single capable AI agent take on real software and product development work while keeping the human in control of the parts that matter. The key levers are the document stack, the use-case granularity, the dependency graph that drives execution, and the explicit evidence trail that makes the work auditable. Future work includes publishing the supporting prompts, templates and examples as a companion repository, and exploring how far the Hoare-triple framing can be pushed toward more formal verification of agent-produced work.

APPENDIX A
STARTER PROMPT FOR APPLYING THE METHOD

The following prompt is intended for a strong code-synthesis agent running in Codex, Claude Code, or a similar development environment. It is deliberately explicit about the document stack, GitHub mapping, evidence trail, and completion criteria.

```
Here is an idea about a system:

<description of idea>

I want you to expand this idea into a coherent document stack, and
then use that document stack to take the idea from this initial
description to a working implementation, provided that the path is
smooth enough to do so without unresolved research questions. Use a
single capable agent; do not simulate a multi-agent organisation.
Treat the work as an auditable software project driven by the document
stack.

Project setup:

- GitHub account or organisation: <github-account>
- Repository name: <repo-slug>
- Local checkout path: ~/git/<repo-slug>
- Public repository URL: https://github.com/<github-account>/<repo-slug>
- Documentation site URL, if this is a web-compatible project:
  https://<github-account>.github.io/<repo-slug>/
- Primary implementation language and platform:
  <language, framework, runtime, and deployment target>

If any value above is missing, infer a conservative default from the
idea and record that assumption in the lab notebook. Ask me only when
the missing value would materially change the product.

Create the repository:

1. Create or reuse ~/git/<repo-slug>.
2. Initialise it as a git repository.
3. Create a public GitHub repository under <github-account>.
4. Push the initial main branch.
5. Add a README that points to the documentation site when it exists.
6. Add a Makefile with useful targets for setup, test, build,
   evidence capture, documentation generation, and publishing.
7. Keep generated artifacts organised; do not leave scripts,
   screenshots, movies, or build products scattered at the top level.

Create and maintain this document stack:

- docs/tldr.md: a one- or two-page summary of the current project.
- docs/concept.md: the idea, motivation, goals, non-goals, audience,
  success criteria, and assumptions.
- docs/prd.md: product requirements from the user perspective,
  including functional requirements, quality requirements, constraints,
  and acceptance criteria.
- docs/architecture.md: the implementation architecture, component
  responsibilities, interfaces, storage, deployment shape, and a
  happy-day scenario. If the architecture is trivial, keep this short
  but still explicit.
- docs/plan.md: the execution plan as simple agentic use cases.
- docs/lab-notebook.md: dated notes from the agent, the human, and
  human decisions communicated through the agent. Record assumptions,
  explorations, surprises, blockers, decisions, and why the plan
  changed.
- docs/evidence.md: an index of evidence showing what was tested,
  captured, reviewed, and accepted.
- evidence/: static evidence pages, screenshots, movies, logs, and
  other inspectable artifacts.
```

````
Keep the stack coherent at all times. When reality changes one
document, update the other documents in the same pass so that the
stack never lies about what is planned, what is known, or what has
been built.

Write the execution plan as a directed acyclic graph of use cases.
Each use case must have:

- a stable id such as UC-01;
- a short title;
- actors;
- pre-conditions;
- post-conditions;
- happy-day steps only;
- observables/evidence required before it can be called done;
- dependencies on earlier use cases;
- current state;
- associated GitHub issue URL;
- associated GitHub PR URL once implementation exists.

Use the following lifecycle colours everywhere the graph is shown:

- green: done and evidence reviewed;
- yellow: ready to run;
- orange: running;
- blue: ready for review;
- red: blocked;
- purple: deferred.

Create GitHub issues from the use cases:

1. Create one GitHub issue per use case.
2. Put the use-case contract in the issue body.
3. Use labels for state, area, and parent/epic relationships.
4. Initially link each graph node to its GitHub issue.
5. For each implemented use case, create a branch and pull request.
6. Put the evidence summary and acceptance checks in the PR body.
7. Once a PR exists, update the graph node so it links to the PR
   instead of the issue.
8. After merge and evidence review, mark the node green.

Maintain docs/plan.md with a Mermaid diagram that starts at the top
and works downward. It must be an acyclic directed graph using
top-to-bottom layout. Make it visually pleasant: short node labels,
stable UC ids, clear phase groupings, enough whitespace, and a small
legend for the lifecycle colours. Use clickable nodes.

Use this Mermaid shape unless the project requires a small variation:

```mermaid
flowchart TB
  classDef done fill:#d9f7d9,stroke:#2f7d32,color:#102a13
  classDef ready fill:#fff6bf,stroke:#9a7b00,color:#2b2300
  classDef running fill:#ffe2bf,stroke:#b45f00,color:#2b1600
  classDef review fill:#d8ecff,stroke:#2d6fb7,color:#102438
  classDef blocked fill:#ffd6d6,stroke:#b3261e,color:#3a0808
  classDef deferred fill:#eadcff,stroke:#6f42c1,color:#20103a

  subgraph P0["Document stack"]
    UC01["UC-01: create TL;DR"]:::done
    UC02["UC-02: write concept"]:::done
    UC03["UC-03: write PRD"]:::done
    UC04["UC-04: write architecture"]:::done
  end

  subgraph P1["Implementation"]
    UC05["UC-05: first runnable slice"]:::ready
````
````

```
    UC06["UC-06: core workflow"]:::ready
    UC07["UC-07: evidence capture"]:::ready
  end

  UC01 --> UC02 --> UC03 --> UC04 --> UC05 --> UC06 --> UC07

  click UC01 "https://github.com/<github-account>/<repo-slug>/issues/1" "Open issue"
```

Do not let the Mermaid diagram become a hairball. If it grows too large, split it into an overview graph plus phase-specific graphs, but keep the overview graph as the canonical top-to-bottom DAG.

Execute the plan:

1. Choose the next use case whose dependencies are complete.
2. Before implementing it, restate its pre-conditions, post-conditions, and evidence requirements in the lab notebook.
3. Implement the smallest coherent slice that satisfies the use case.
4. Run the relevant tests, linters, build commands, and smoke checks.
5. Capture evidence. For web or graphical systems, use Playwright to capture screenshots and short mp4 movies of the system in use. Do not play games or workflows to completion unless completion itself is the evidence requirement; capture enough to show the behaviour works in real time.
6. Add or update static evidence pages under evidence/ so that a human can browse the project's evolution.
7. Update the document stack, the Mermaid graph, GitHub issue/PR links, and the lab notebook.
8. Commit, push, open a PR, and record the PR in the graph.
9. Merge only when the evidence satisfies the use-case contract.

Blocking rule:

If a use case stops being a smooth implementation task because of an unknown algorithm, missing credential, external service failure, ambiguous product decision, or research problem, mark it red as blocked. Update docs/plan.md, docs/lab-notebook.md, and the GitHub issue with the exact blocker, what was tried, and what human decision or external change is needed. Do not silently invent facts, APIs, references, credentials, or product decisions.

Testing and evidence expectations:

- Include automated tests appropriate to the project.
- Include at least one smoke test of the final user-visible workflow.
- Keep local, repeatable commands in the Makefile.
- For browser projects, include Playwright tests and Playwright-based evidence capture.
- Store screenshots and movies with stable names.
- Build a static evidence index that shows the product over time.
- Record what each screenshot, movie, or log proves.

Documentation site:

If the project can reasonably publish static documentation, create a documentation site and publish it with GitHub Pages. The first page should present the project clearly, link to the TL;DR, concept, PRD, architecture, plan, lab notebook, evidence index, GitHub repository, and playable or runnable product where applicable. It should look like a polished project presentation, not just a directory listing.

Final product:

Implement the actual system described by the idea, not only the documentation. The final repository should contain:

```
- the working implementation;
- tests and build scripts;
- the complete coherent document stack;
- the lab notebook;
- the Mermaid use-case DAG with issue/PR links;
- GitHub issues and PRs corresponding to the use cases;
- captured evidence, including screenshots and movies when relevant;
- a public GitHub repository;
- a published documentation site when applicable;
- clear instructions for running, testing, and publishing.

Completion criteria:

The work is complete only when the implementation runs, the tests pass,
the evidence is inspectable, the docs describe the actual implemented
system, the graph is green for completed use cases, GitHub links point
to the relevant issues or PRs, and the README tells a new reader where
to find the site, the docs, and the runnable product.
```

REFERENCES

[1] A. Cockburn, *Writing Effective Use Cases*, ser. Agile Software Development Series. Addison-Wesley Professional, October 2000, verified: InformIT publisher page accessible; ISBN-13 978-0-201-70225-5 confirmed. Standard reference for use-case methodology. [Online]. Available: https://www.informit.com/store/writing-effective-use-cases-9780201702255

[2] C. A. R. Hoare, "An axiomatic basis for computer programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, October 1969, verified: DOI 10.1145/363235.363259 resolves to ACM Digital Library record. Foundational paper introducing Hoare triples and axiomatic semantics. [Online]. Available: https://dl.acm.org/doi/10.1145/363235.363259

[3] Microsoft, "Playwright: Reliable end-to-end testing for modern web apps," Official project documentation, 2024, verified: Official Playwright documentation site accessible. Cross-browser automation framework also used for AI-agent web interaction. [Online]. Available: https://playwright.dev/

[4] Mermaid Contributors, "Mermaid: Diagramming and charting tool," Open-source project documentation, 2024, verified: Official Mermaid documentation site accessible. Open-source JavaScript library for text-based diagram generation. [Online]. Available: https://mermaid.js.org/

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023, verified: arXiv abstract page accessible; authors, title and ICLR 2023 publication confirmed. Foundational paper on interleaved reasoning and acting in LLM agents. [Online]. Available: https://arxiv.org/abs/2210.03629

[6] E. Schluntz and B. Zhang, "Building effective agents," Anthropic engineering blog, December 2024, verified: URL accessible; authorship and December 19, 2024 publication date confirmed on Anthropic engineering site. Practitioner-oriented overview of agent design patterns. [Online]. Available: https://www.anthropic.com/engineering/building-effective-agents